

Semana 7:

React Defensivo

Manejo de Errores, Estados de Carga y Seguridad para Apps a Prueba de Fallos.



El Mito del Happy Path

El Camino Feliz



Las redes fallan.
Las APIs se caen.
Los usuarios son impredecibles.



Capa Superficial: UX y Feedback
(Estados de Carga)

Capa Lógica: Resiliencia
(Manejo de Errores)

Capa Profunda: Integridad
(Seguridad y XSS)

Un desarrollador junior asume que todo funcionará.
Un profesional diseña para el fallo.

Capa 1: UX Visual y Estados de Carga

En la asincronía, la latencia es inevitable. Sin feedback visual, el usuario asume que la app está colgada.



Implementando el Estado de Carga

Anatomy of Code

El estado local controla la UI.

```
1  
2  
3  const [isLoading, setIsLoading] = useState(false);  
4  
5  <button disabled={isLoading}>  
6    {isLoading ? <Spinner /> : 'Enviar'}  
7  </button>
```

Renderizado condicional instantáneo.

Bloquea interacciones duplicadas mientras la petición viaja.

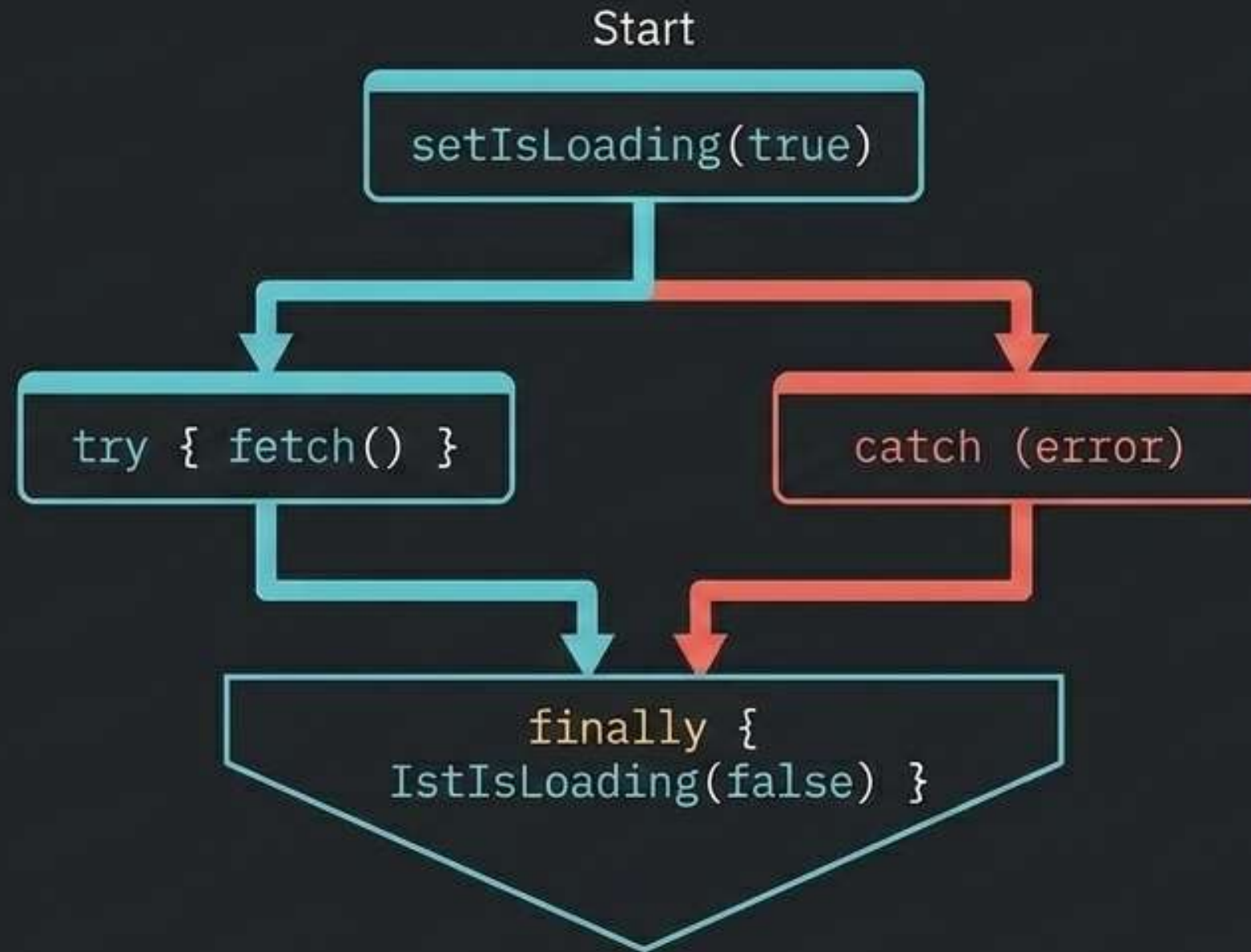
Capa 2: Resiliencia ante Excepciones



La asincronía es peligrosa. Las promesas HTTP no garantizan una respuesta exitosa. Si la red falla y no hay una red de seguridad, la aplicación colapsa.

Nuestra red de seguridad es el bloque try / catch.

El Flujo de Vida de una Petición



El bloque `finally` garantiza que, pase lo que pase (éxito o desastre), el spinner de carga se apagará. Adiós a las pantallas de cargando infinito.

Anatomía del Try/Catch con Fetch

```
try {  
  const response = await fetch('/api/data');  
  if (!response.ok) throw new Error('Fallo en API');  
} catch (error) {  
  setError(error.message);  
}
```

¡Trampa de Fetch!

fetch solo lanza excepciones automáticamente ante fallos de red completos.

Debemos forzar el error manualmente si el servidor responde con un código 400 o 500.

Matriz de Reacción: Códigos HTTP vs. Interfaz

 200s (Éxito)	200 OK	Acción UI: Renderizar datos recibidos.
 400s (Cliente)	400 Bad Request / 404 Not Found	Acción UI: Mostrar mensaje amigable al usuario ("Revisa tus datos" o "No encontramos lo que buscas").
 500s (Servidor)	500 Internal Error	Acción UI: Desplegar pantalla de error global o Toast de emergencia ("Nuestros servidores están teniendo problemas").

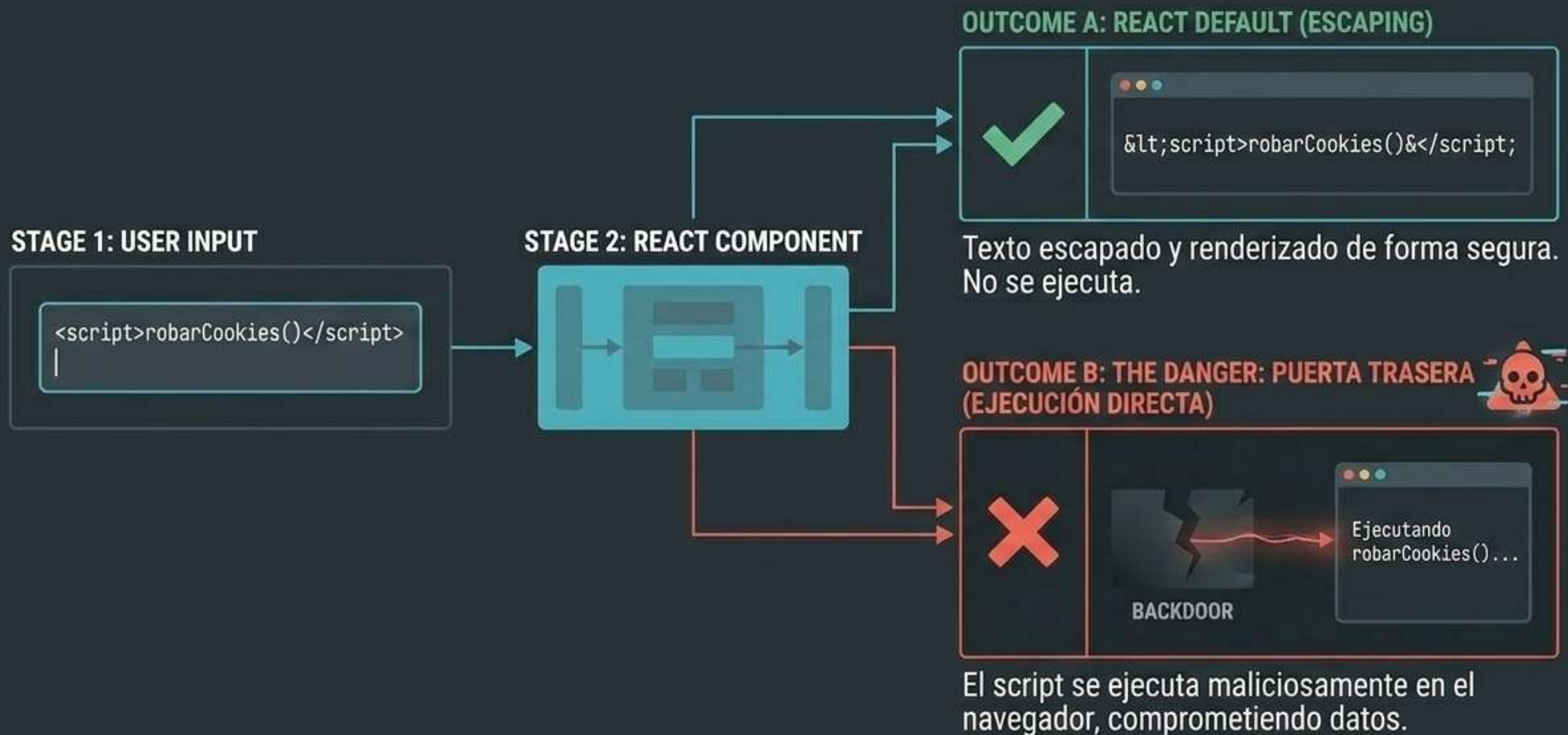
Capa 3: Seguridad y el Lado Oscuro



Manejar errores protege la app de sí misma. La seguridad la protege del exterior.
El Frontend es un entorno hostil; la regla de oro es:

Nunca confíes en los datos del usuario.

La Trampa del XSS (Cross-Site Scripting)



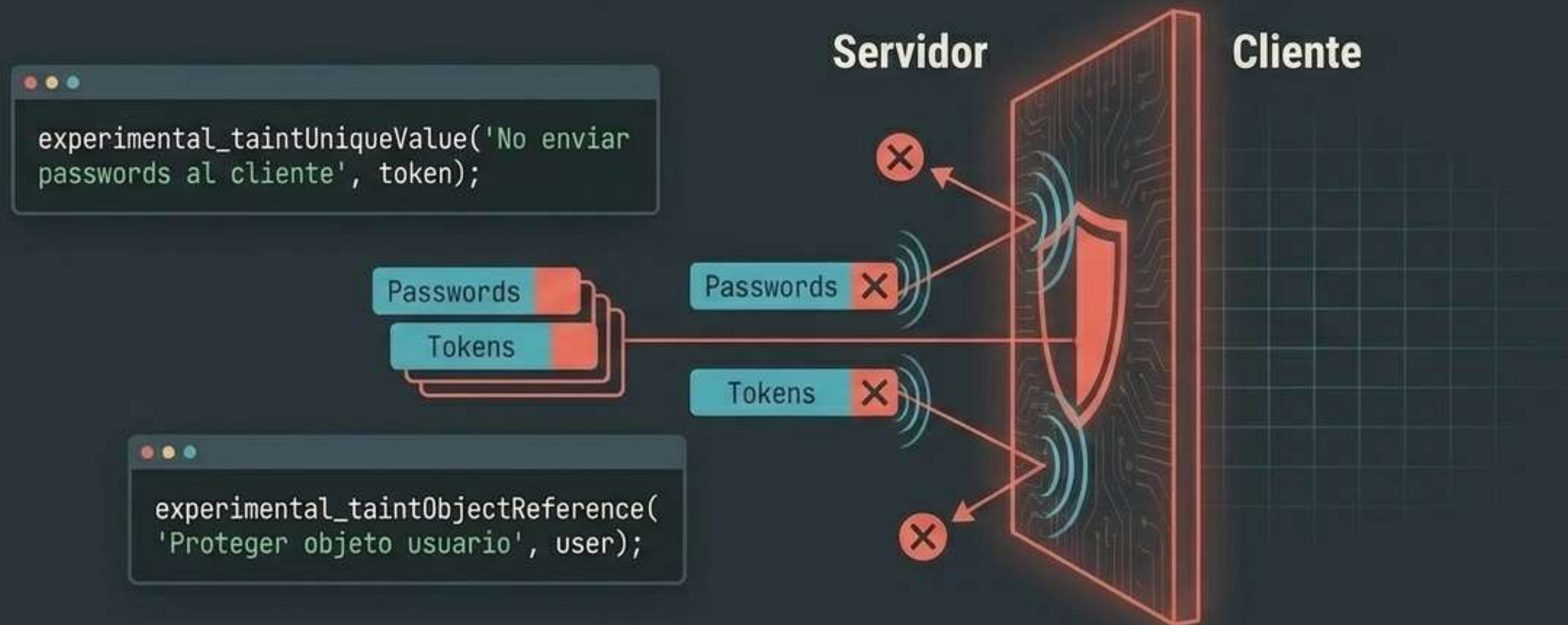
Cuidado con dangerouslySetInnerHTML



```
<div dangerouslySetInnerHTML={{__html: comentarioUsuario}} />
```

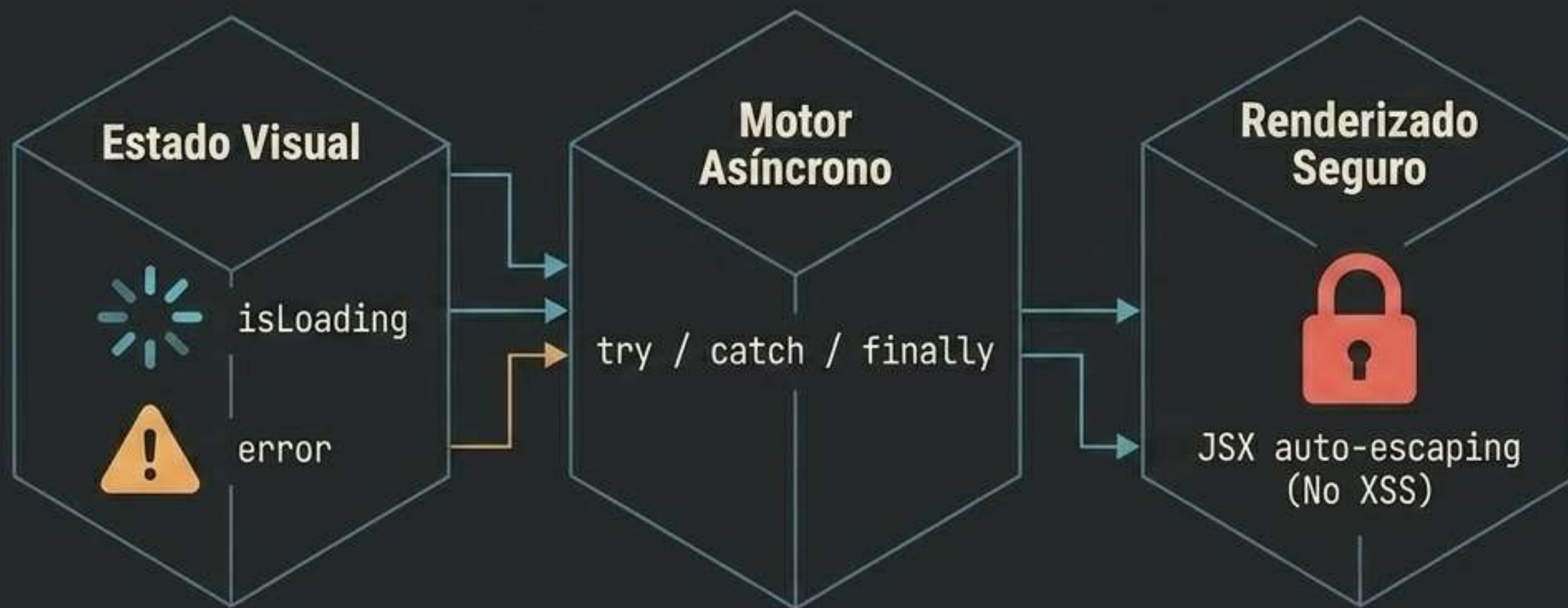
- * El nombre es una advertencia literal. Usar esta propiedad anula la protección nativa de React contra XSS.
- * NUNCA la utilices para renderizar contenido no sanitizado proveniente de usuarios o API externas no confiables.

Prevención de Fugas de Datos (Taint APIs)



Las APIs modernas de React (Taint) permiten 'marcar' datos confidenciales. El sistema lanza un error de inmediato si intentan filtrarse hacia el cliente.

Síntesis: El Componente Antibalas



Un vistazo al ecosistema perfecto. El código no solo funciona: informa al usuario (UX), se recupera de fallos de red (Resiliencia) y bloquea inyecciones (Seguridad).

Próximos Pasos

- ☒ Implementar `spinners` vinculados al estado.
- ☒ Capturar y manejar respuestas 400/500 con `try/catch`.
- ☒ Sanitizar inputs y evitar renderizado peligroso.

Tu código ya no solo asume el 'Camino Feliz'.
Estás listo para el Laboratorio. Protege tu SPA.